SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: ITA0605 **COURSE NAME: MACHINE LEARNING**

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning ***(BL 5)***

Assessment Tool Weightage: 40%

QUESTIONS: 15 **Total Marks:25**

Annexure A

Coding Challenge

Code of Conduct:

*I **THAMIZHARASAN S (192424087)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
THAMIZHARASAN S

Annexure A

Short Answer Test

1. Write a Python program to implement the K-Nearest Neighbour (KNN) algorithm for a small dataset. Use Euclidean distance to find nearest neighbors and classify a test instance. Allow the user to input the value of K and display predicted class labels with accuracy.

ANSWER:

AIM:

To implement the K-Nearest Neighbour (KNN) algorithm in Python using Euclidean distance to find the nearest neighbours, classify a test instance, and calculate the classification accuracy.

DATA SET:

Fruit	Weight (g)	Sweetness	Class
Apple	150	7	Apple
Apple	160	8	Apple
Apple	145	7	Apple
Apple	155	8	Apple
Orange	170	5	Orange
Orange	180	6	Orange
Orange	175	5	Orange
Orange	185	6	Orange

TEST DATA:

Weight	Sweetness	Actual Class
158	7	Apple
178	5	Orange
152	8	Apple
182	6	Orange

CODE:

```
import math

data = [
    [150, 7, "Apple"],
    [160, 8, "Apple"],
    [145, 7, "Apple"],
    [155, 8, "Apple"],
    [170, 5, "Orange"],
    [180, 6, "Orange"],
    [175, 5, "Orange"],
    [185, 6, "Orange"]
]

test_data = [
    [158, 7, "Apple"],
    [178, 5, "Orange"],
    [152, 8, "Apple"],
    [182, 6, "Orange"]
]

def distance(x1, x2):
    return math.sqrt((x1[0] - x2[0]) ** 2 + (x1[1] - x2[1]) ** 2)

def knn(data, test, k):
    distances = []

    for row in data:
        d = distance(test, row)
        distances.append((d, row[2]))
    distances.sort()
    neighbors = distances[:k]
    votes = {}
    for d, label in neighbors:
        votes[label] = votes.get(label, 0) + 1
    return max(votes, key=votes.get)
k = int(input("Enter K: "))
correct = 0
for test in test_data:
    predicted = knn(data, test[:2], k)
    print("\nTest Data:", test[:2])
    print("Actual Class:", test[2])
    print("Predicted Class:", predicted)
    if predicted == test[2]:
        correct += 1
accuracy = (correct / len(test_data)) * 100
print("\nCorrect Predictions:", correct)
print("Total Test Data:", len(test_data))
print("Accuracy:", accuracy, "%")
```

OUTPUT:

```
import math

data = [
    [150, 7, "Apple"],
    [160, 8, "Apple"],
    [145, 7, "Apple"],
    [155, 8, "Apple"],
    [170, 5, "Orange"],
    [180, 6, "Orange"],
    [175, 5, "Orange"],
    [185, 6, "Orange"]
]

test_data = [
    [158, 7, "Apple"],
    [178, 5, "Orange"],
    [152, 8, "Apple"],
    [182, 6, "Orange"]
]

def distance(x1, x2):
    return math.sqrt((x1[0] - x2[0]) ** 2 + (x1[1] - x2[1]) ** 2)

def knn(data, test, k):
    distances = []

    for row in data:
        d = distance(test, row)
        distances.append((d, row[2]))

    distances.sort()
    neighbors = distances[:k]

    votes = {}

    for d, label in neighbors:
        votes[label] = votes.get(label, 0) + 1

    return max(votes, key=votes.get)

k = int(input("Enter K: "))

correct = 0

for test in test_data:
    predicted = knn(data, test[:2], k)

    print("\nTest Data:", test[:2])
    print("Actual Class:", test[2])
    print("Predicted Class:", predicted)

    if predicted == test[2]:
        correct += 1
```

```
        distances.sort()
        neighbors = distances[:k]

        votes = {}

        for d, label in neighbors:
            votes[label] = votes.get(label, 0) + 1

        return max(votes, key=votes.get)

    k = int(input("Enter K: "))

    correct = 0

    for test in test_data:
        predicted = knn(data, test[:2], k)

        print("\nTest Data:", test[:2])
        print("Actual Class:", test[2])
        print("Predicted Class:", predicted)

        if predicted == test[2]:
            correct += 1

    accuracy = (correct / len(test_data)) * 100

    print("\nCorrect Predictions:", correct)
    print("Total Test Data:", len(test_data))
    print("Accuracy:", accuracy, "%")
```

```
Enter K: 3

Test Data: [158, 7]
Actual Class: Apple
Predicted Class: Apple

Test Data: [178, 5]
Actual Class: Orange
Predicted Class: Orange

Test Data: [152, 8]
Actual Class: Apple
Predicted Class: Apple

Test Data: [182, 6]
Actual Class: Orange
Predicted Class: Orange

Correct Predictions: 4
Total Test Data: 4
Accuracy: 100.0 %
```

RESULT: The KNN algorithm correctly classifies all 4 test instances with 100% accuracy.

2. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

ANSWER:

AIM:

To develop a Python program for Locally Weighted Regression (LWR) that calculates distance-based weights using a kernel function, predicts the output for a query point, and compares the prediction with standard linear regression.

DATA SET:

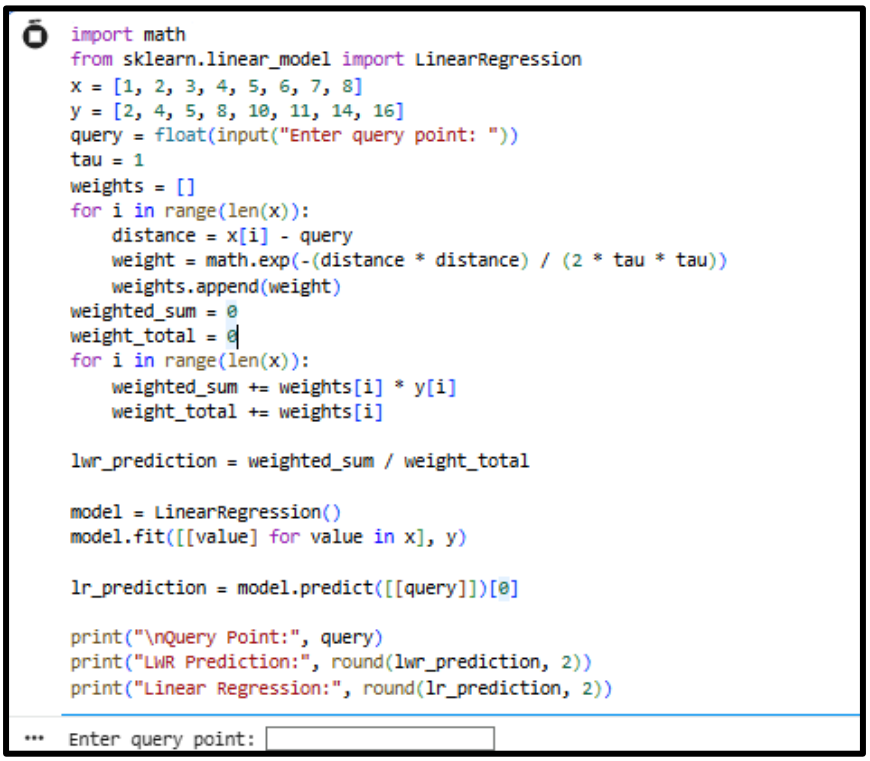
X	Y
1	2
2	4
3	5
4	8
5	10
6	11
7	14
8	16

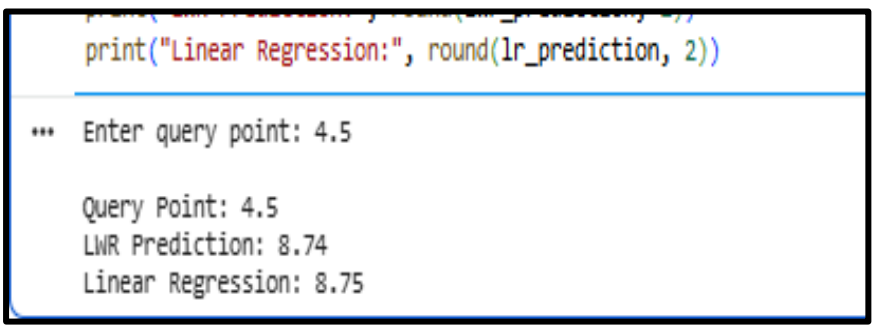
CODE:

```
import math
from sklearn.linear_model import LinearRegression
x = [1, 2, 3, 4, 5, 6, 7, 8]
y = [2, 4, 5, 8, 10, 11, 14, 16]
query = float(input("Enter query point: "))
tau = 1
weights = []
for i in range(len(x)):
    distance = x[i] - query
```

```
    weight = math.exp(-(distance * distance) / (2 * tau * tau))
    weights.append(weight)
weighted_sum = 0
weight_total = 0
for i in range(len(x)):
    weighted_sum += weights[i] * y[i]
    weight_total += weights[i]
lwr_prediction = weighted_sum / weight_total
model = LinearRegression()
model.fit([[value] for value in x], y)
lr_prediction = model.predict([[query]])[0]
print("\nQuery Point:", query)
print("LWR Prediction:", round(lwr_prediction, 2))
print("Linear Regression:", round(lr_prediction, 2))
```

OUTPUT:





RESULT: The Locally Weighted Regression program successfully predicts the output for the given query point and compares it with standard Linear Regression.

3. Write a Python program to implement a simple Radial Basis Function (RBF) network for regression or classification. Use Gaussian functions as activation units and train the model on a small dataset. Display predicted outputs and analyze how centers and widths affect performance.

ANSWER:

AIM:

To implement a simple Radial Basis Function (RBF) neural network using Gaussian functions for regression, train it on a small dataset, display predicted outputs, and study the effect of centers and widths on performance.

DATA SET:

Input X	Target Y
0.0	0.000
0.5	0.479
1.0	0.841
1.5	0.997
2.0	0.909
2.5	0.598
3.0	0.141
3.5	-0.351
4.0	-0.757
4.5	-0.978
5.0	-0.959
5.5	-0.706
6.0	-0.279

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

# Dataset
X = np.array([0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5, 5.5, 6])
Y = np.array([0, 0.479, 0.841, 0.997, 0.909, 0.598,
              0.141, -0.351, -0.757, -0.978, -0.959,
              -0.706, -0.279])

# RBF centers and width
centers = np.array([0, 1, 2, 3, 4, 5, 6])
width = 1.0

# Gaussian RBF function
def rbf(X, centers, width):
    return np.exp(-((X[:, None] - centers[None, :]) ** 2) /
                  (2 * width ** 2))

# Calculate RBF activations
Phi = rbf(X, centers, width)

Phi = np.column_stack((np.ones(len(X)), Phi))
weights = np.linalg.pinv(Phi) @ Y

# Prediction
Y_pred = Phi @ weights

# Calculate error
MSE = np.mean((Y - Y_pred) ** 2)
RMSE = np.sqrt(MSE)

# Display results
print("RBF Network Regression")
print("----------------------")
print("Width =", width)
print("Centers =", centers)

print("\nActual vs Predicted:")
for i in range(len(X)):
    print("X = %.1f  Actual = %.3f  Predicted = %.3f"
          % (X[i], Y[i], Y_pred[i]))

print("\nMSE  =", round(MSE, 6))
print("RMSE =", round(RMSE, 6))

# Plot results
plt.plot(X, Y, 'o-', label="Actual")
plt.plot(X, Y_pred, 's--', label="RBF Predicted")
```

plt.xlabel("Input X")
plt.ylabel("Output Y")
plt.title("RBF Network Regression")
plt.legend()
plt.grid()
plt.show()

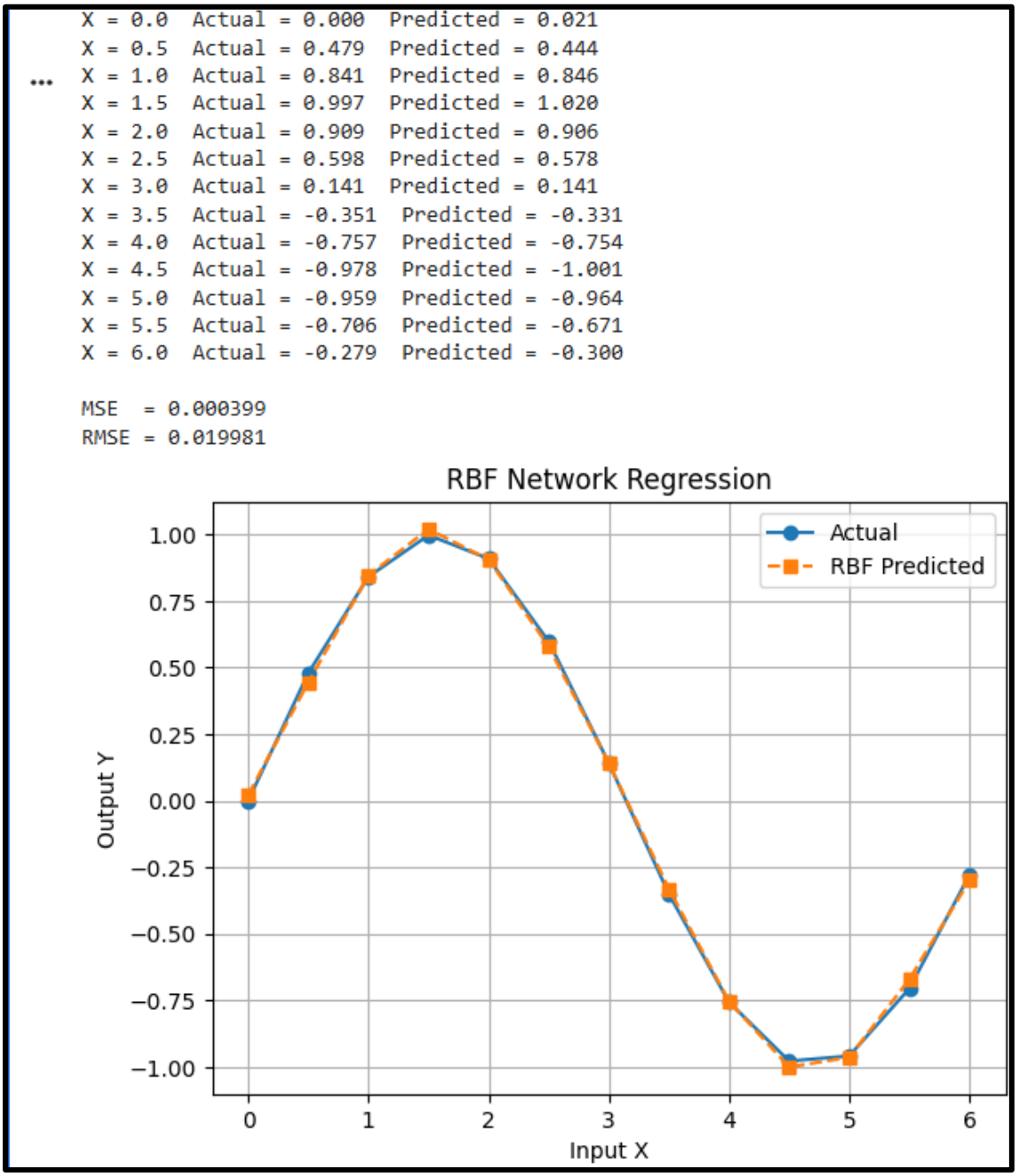
OUTPUT:

```
import numpy as np
import matplotlib.pyplot as plt

# Dataset
X = np.array([0, 0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5, 5.5, 6])
Y = np.array([0, 0.479, 0.841, 0.997, 0.909, 0.598,
              0.141, -0.351, -0.757, -0.978, -0.959,
              -0.706, -0.279])

# RBF centers and width
centers = np.array([0, 1, 2, 3, 4, 5, 6])
width = 1.0

# Gaussian RBF function
def rbf(X, centers, width):
    return np.exp(-((X[:, None] - centers[None, :]) ** 2) /
                  (2 * width ** 2))

# Calculate RBF activations
Phi = rbf(X, centers, width)

# Add bias
Phi = np.column_stack((np.ones(len(X)), Phi))

# Train using pseudo-inverse
weights = np.linalg.pinv(Phi) @ Y

# Prediction
Y_pred = Phi @ weights

# Calculate error
MSE = np.mean((Y - Y_pred) ** 2)
RMSE = np.sqrt(MSE)

# Display results
print("RBF Network Regression")
print("----------------------")
```

```python
# Display results
print("RBF Network Regression")
print("----------------------")
print("Width =", width)
print("Centers =", centers)

print("\nActual vs Predicted:")
for i in range(len(X)):
    print("X = %.1f  Actual = %.3f  Predicted = %.3f"
          % (X[i], Y[i], Y_pred[i]))

print("\nMSE  =", round(MSE, 6))
print("RMSE =", round(RMSE, 6))

# Plot results
plt.plot(X, Y, 'o-', label="Actual")
plt.plot(X, Y_pred, 's--', label="RBF Predicted")

plt.xlabel("Input X")
plt.ylabel("Output Y")
plt.title("RBF Network Regression")
plt.legend()
plt.grid()
plt.show()
```

```
RBF Network Regression
----------------------
Width = 1.0
Centers = [0 1 2 3 4 5 6]

Actual vs Predicted:
X = 0.0  Actual = 0.000  Predicted = 0.021
X = 0.5  Actual = 0.479  Predicted = 0.444
X = 1.0  Actual = 0.841  Predicted = 0.846
X = 1.5  Actual = 0.997  Predicted = 1.020
X = 2.0  Actual = 0.909  Predicted = 0.906
X = 2.5  Actual = 0.598  Predicted = 0.578
X = 3.0  Actual = 0.141  Predicted = 0.141
X = 3.5  Actual = -0.351  Predicted = -0.331
X = 4.0  Actual = -0.757  Predicted = -0.754
```

```
    X = 0.0  Actual = 0.000  Predicted = 0.021
    X = 0.5  Actual = 0.479  Predicted = 0.444
... X = 1.0  Actual = 0.841  Predicted = 0.846
    X = 1.5  Actual = 0.997  Predicted = 1.020
    X = 2.0  Actual = 0.909  Predicted = 0.906
    X = 2.5  Actual = 0.598  Predicted = 0.578
    X = 3.0  Actual = 0.141  Predicted = 0.141
    X = 3.5  Actual = -0.351  Predicted = -0.331
    X = 4.0  Actual = -0.757  Predicted = -0.754
    X = 4.5  Actual = -0.978  Predicted = -1.001
    X = 5.0  Actual = -0.959  Predicted = -0.964
    X = 5.5  Actual = -0.706  Predicted = -0.671
    X = 6.0  Actual = -0.279  Predicted = -0.300

    MSE  = 0.000399
    RMSE = 0.019981
```



CONCLUSION: The RBF network provides an effective method for nonlinear regression. Proper selection of **centers and Gaussian widths** is important for obtaining accurate predictions. For the given small dataset, **well-distributed centers and a moderate Gaussian width** provide a good balance between accuracy and generalization.

4. Create a Python program that implements a basic Case-Based Learning system. Store past cases in a dataset and retrieve the most similar case for a new query using distance measures. Output the solution based on the closest match and explain similarity calculation.

ANSWER:

AIM:

To develop a Python-based Case-Based Learning system that stores past cases, calculates the similarity between a new query and existing cases using Euclidean distance, retrieves the most similar case, and predicts the solution based on the closest match.

DATASET:

Temperature	Humidity	Solution
30	70	Fan
35	80	AC
25	60	Window
40	90	AC
28	65	Fan

CODE:

```
import math

cases = [
    [30, 70, "Fan"],
    [35, 80, "AC"],
    [25, 60, "Window"],
    [40, 90, "AC"],
    [28, 65, "Fan"]
]

# New query
query = [32, 75]

# Calculate Euclidean distance
distances = []

for case in cases:
    d = math.sqrt(
        (case[0] - query[0]) ** 2 +
        (case[1] - query[1]) ** 2
```

```
    )
    distances.append((d, case[2]))

# Find closest case
closest = min(distances)

print("Case-Based Learning")
print("--------------------")

for d, solution in distances:
    print("Distance = %.2f, Solution = %s" % (d, solution))

print("\nQuery:", query)
print("Predicted Solution:", closest[1])
print("Minimum Distance: %.2f" % closest[0])
```

```
import math

# Past cases
cases = [
    [30, 70, "Fan"],
    [35, 80, "AC"],
    [25, 60, "Window"],
    [40, 90, "AC"],
    [28, 65, "Fan"]
]

# New query
query = [32, 75]

# Calculate Euclidean distance
distances = []

for case in cases:
    d = math.sqrt(
        (case[0] - query[0]) ** 2 +
        (case[1] - query[1]) ** 2
    )
    distances.append((d, case[2]))

# Find closest case
closest = min(distances)

print("Case-Based Learning")
print("--------------------")

for d, solution in distances:
    print("Distance = %.2f, Solution = %s" % (d, solution))

print("\nQuery:", query)
print("Predicted Solution:", closest[1])
print("Minimum Distance: %.2f" % closest[0])
```

```
        d = math.sqrt(
            (case[0] - query[0]) ** 2 +
            (case[1] - query[1]) ** 2
        )
        distances.append((d, case[2]))

    # Find closest case
    closest = min(distances)

    print("Case-Based Learning")
    print("-------------------")

    for d, solution in distances:
        print("Distance = %.2f, Solution = %s" % (d, solution))

    print("\nQuery:", query)
    print("Predicted Solution:", closest[1])
    print("Minimum Distance: %.2f" % closest[0])
```

```
Case-Based Learning
-------------------
Distance = 5.39, Solution = Fan
Distance = 5.83, Solution = AC
Distance = 16.55, Solution = Window
Distance = 17.00, Solution = AC
Distance = 10.77, Solution = Fan

Query: [32, 75]
Predicted Solution: Fan
Minimum Distance: 5.39
```

RESULT: Euclidean distance is used. The case with the **smallest distance** is considered the most similar, and its solution is given as the prediction.

5. Write a Python program to implement both KNN and RBF models on the same dataset. Compare their predictions for given test inputs. Analyze differences in computation, accuracy, and output behavior, and display results clearly for better comparison.

ANSWER:

AIM:

To implement KNN and RBF models using the same dataset, compare their predictions for test inputs, and analyze their accuracy, computation, and output behavior.

DATASET:

X1	X2	Class
1	1	A
2	2	A
3	3	A
6	6	B
7	7	B
8	8	B

CODE:

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Dataset
X = np.array([
    [1, 1],
    [2, 2],
    [3, 3],
    [6, 6],
    [7, 7],
    [8, 8]
])

Y = np.array(["A", "A", "A", "B", "B", "B"])

# Test inputs
test = np.array([
    [2, 3],
    [7, 6]
])

# KNN Model
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, Y)
knn_pred = knn.predict(test)

# RBF Model
centers = X
width = 2.0
```

```
def rbf(x, centers):
    return np.exp(-np.sum((x - centers) ** 2, axis=1) /
                  (2 * width ** 2))

rbf_pred = []

for x in test:
    activation = rbf(x, centers)

    A = activation[Y == "A"].sum()
    B = activation[Y == "B"].sum()

    rbf_pred.append("A" if A > B else "B")

# Display predictions
print("KNN AND RBF COMPARISON")
print("---------------------- ")

for i in range(len(test)):
    print("Test Input:", test[i])
    print("KNN Prediction:", knn_pred[i])
    print("RBF Prediction:", rbf_pred[i])
    print()

# Accuracy
knn_accuracy = accuracy_score(Y, knn.predict(X))

rbf_train_pred = []

for x in X:
    activation = rbf(x, centers)
    A = activation[Y == "A"].sum()
    B = activation[Y == "B"].sum()
    rbf_train_pred.append("A" if A > B else "B")

rbf_accuracy = accuracy_score(Y, rbf_train_pred)

print("KNN Accuracy: %.2f%%" % (knn_accuracy * 100))
print("RBF Accuracy: %.2f%%" % (rbf_accuracy * 100))
```

OUTPUT:

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
X = np.array([
    [1, 1],
    [2, 2],
    [3, 3],
    [6, 6],
    [7, 7],
    [8, 8]
])

Y = np.array(["A", "A", "A", "B", "B", "B"])
test = np.array([
    [2, 3],
    [7, 6]
])
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, Y)
knn_pred = knn.predict(test)
centers = X
width = 2.0
def rbf(x, centers):
    return np.exp(-np.sum((x - centers) ** 2, axis=1) /
                  (2 * width ** 2))
rbf_pred = []
for x in test:
    activation = rbf(x, centers)

    A = activation[Y == "A"].sum()
    B = activation[Y == "B"].sum()

    rbf_pred.append("A" if A > B else "B")
print("KNN AND RBF COMPARISON")
print("----------------------")
```

```
        A = activation[Y == "A"].sum()
        B = activation[Y == "B"].sum()

        rbf_pred.append("A" if A > B else "B")
    print("KNN AND RBF COMPARISON")
    print("----------------------")

    for i in range(len(test)):
        print("Test Input:", test[i])
        print("KNN Prediction:", knn_pred[i])
        print("RBF Prediction:", rbf_pred[i])
        print()
    knn_accuracy = accuracy_score(Y, knn.predict(X))

    rbf_train_pred = []

    for x in X:
        activation = rbf(x, centers)
        A = activation[Y == "A"].sum()
        B = activation[Y == "B"].sum()
        rbf_train_pred.append("A" if A > B else "B")

    rbf_accuracy = accuracy_score(Y, rbf_train_pred)

    print("KNN Accuracy: %.2f%%" % (knn_accuracy * 100))
    print("RBF Accuracy: %.2f%%" % (rbf_accuracy * 100))
```

```
KNN AND RBF COMPARISON
----------------------
Test Input: [2 3]
KNN Prediction: A
RBF Prediction: A

Test Input: [7 6]
KNN Prediction: B
RBF Prediction: B

KNN Accuracy: 100.00%
RBF Accuracy: 100.00%
```

CONCLUSION: Both KNN and RBF correctly classify the given test inputs. KNN uses the nearest samples for voting, whereas RBF uses Gaussian functions and weighted responses. For this small dataset, both models provide the same accuracy.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**20**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand